

ASSIGNMENT

ONLINEBOOKSTORE

Database creates

```
create database onlinebookstore;
```

Create tables

```
create table Books (
```

```
Book_ID serial primary key,
```

```
Title varchar(100),
```

```
Author varchar(100),
```

```
Genre varchar(50),
```

```
Published_Year int,
```

```
Price numeric(10,2),
```

```
Stock int
```

```
);
```

```
select * from books;
```

copy

```
Books (Book_ID, Title, Author, Genre, Published_Year, Price, Stock )
```

```
from 'D:\ST - SQL ALL PRACTICE FILES SD61 (1)\ST - SQL ALL PRACTICE FILES-2\All Excel Practice  
Files\Books.csv'
```

```
delimiter ','
```

```
csv header;
```

```
create table customers(
```

```
Customer_ID serial primary key,
```

```
Name varchar(50),
```

```
Email varchar(50) unique,
```

```
Phone varchar (11),
```

```
City varchar(100),
```

ASSIGNMENT ONLINEBOOKSTORE

Country varchar(100)

);

select * from customers;

copy

customers

(Customer_ID, Name, Email, Phone, City, Country)

from 'D:\ST - SQL ALL PRACTICE FILES SD61 (1)\ST - SQL ALL PRACTICE FILES-2\All Excel Practice Files\Customers.csv'

delimiter ','

csv header;

drop table if exists orders;

create table orders(

Order_ID serial primary key,

Customer_ID int references customers (Customer_ID),

Book_ID int references books (Book_ID),

Order_Date date,

Quantity int,

Total_Amount numeric (10,2)

);

select * from orders;

copy

orders(Order_ID, Customer_ID, Book_ID, Order_Date, Quantity, Total_Amount)

from 'D:\ST - SQL ALL PRACTICE FILES SD61 (1)\ST - SQL ALL PRACTICE FILES-2\All Excel Practice Files\Orders.csv'

ASSIGNMENT ONLINEBOOKSTORE

delimiter ','

csv header;

-- 1) Retrieve all books in the "Fiction" genre:

```
select * from books
```

```
where genre = 'Fiction';
```

-- 2) Find books published after the year 1950:

```
select * from books
```

```
where published_year > 1950;
```

-- 3) List all customers from the Canada:

```
select * from customers
```

```
where country = 'Canada';
```

-- 4) Show orders placed in November 2023:

```
select * from orders
```

```
where order_date between '2023-11-01' and '2023-11-30';
```

-- 5) Retrieve the total stock of books available:

```
select sum(stock) as total_stock
```

```
from books;
```

-- 6) Find the details of the most expensive book:

```
select * from books
```

```
order by price desc limit 1;
```

ASSIGNMENT ONLINEBOOKSTORE

-- 7) Show all customers who ordered more than 1 quantity of a book:

```
select * from orders
```

```
where quantity>1;
```

-- 8) Retrieve all orders where the total amount exceeds \$20:

```
select * from orders
```

```
where total_amount >20;
```

-- 9) List all genres available in the Books table:

```
select distinct genre from books;
```

-- 10) Find the book with the lowest stock:

```
select * from books
```

```
order by stock limit 1 ;
```

-- 11) Calculate the total revenue generated from all orders:

```
select sum(total_amount) as reveueu
```

```
from orders;
```

-- 1) Retrieve the total number of books sold for each genre:

```
select b.genre,sum(o.quantity) as total_book_sold
```

```
from books b
```

```
join orders o
```

```
on b.Book_ID = o.Book_ID
```

```
group by b.genre;
```

-- 2) Find the average price of books in the "Fantasy" genre:

```
SELECT *
```

ASSIGNMENT ONLINEBOOKSTORE

FROM books;

select Avg(price) as Avg_Price

from books

where genre ='Fantasy';

-- 3) List customers who have placed at least 2 orders:

select * from orders;

select * from customers;

select o.customer_id, c.name, count(o.order_id) as totals_order

from orders o

join customers c

on c.customer_id = o.customer_id

group by o.customer_id, c.name

having count(o.order_id) >= 2;

select c.customer_id, c.name, count(o.order_id) as totals_order

from orders o

join customers c

on c.customer_id = o.customer_id

group by c.customer_id, c.name

having count(o.order_id) >= 2;

-- 4) Find the most frequently ordered book:

select o.book_id, b.title, count(o.order_id) as ordered

ASSIGNMENT ONLINEBOOKSTORE

```
from orders o
join books b
on o.book_id = b.book_id
group by o.book_id, b.title
order by ordered desc limit 1;
```

-- 5) Show the top 3 most expensive books of 'Fantasy' Genre :

```
select * from books
where genre ='Fantasy'
order by price desc limit 3;
```

-- 6) Retrieve the total quantity of books sold by each author:

```
select b.author, sum(o.quantity) as total_book_sold
from books b
join orders o
on b.book_id = o.book_id
group by b.author;
```

-- 7) List the cities where customers who spent over \$30 are located:

```
select distinct c.city, o.total_amount
from customers c
join orders o
on c.customer_id = o.customer_id
where total_amount > 30
```

ASSIGNMENT

ONLINEBOOKSTORE

order by total_amount desc;

-- 8) Find the customer who spent the most on orders:

```
select c.customer_id, c.name, sum(o.total_amount) as total_spent
from customers c
join orders o
on c.customer_id = o.customer_id
group by c.customer_id
order by total_spent desc limit 1;
```

--9) Calculate the stock remaining after fulfilling all orders:

```
select b.book_id, b.title, b.stock ,coalesce(sum(o.quantity),0) as ordered_quantity,
b.stock- coalesce(sum(o.quantity),0) as balanced
from books b
left join orders o
on b.book_id = o.book_id
group by b.book_id
order by b.book_id;
```